Exception Handling in Java – Practice Sheet - 6	
Aim:	To understand and practice Java exception handling using try, catch, finally, throw, throws, custom exceptions, nested try blocks, multiple catch blocks, exception propagation and try-with-resources.
Tools/Environment:	JDK 17 or later / Visual Studio Code / Command Prompt or Terminal
Prerequisites:	Basic Java syntax, variables, operators, decision making, loops, methods, arrays, classes and objects
Practice Sheet – Theory and Solved Questions	

Part A – Theory

1. Exception Handling in Java

An exception is an event that interrupts the normal flow of program execution. Java provides exception handling so that a program can detect an abnormal condition, handle it appropriately and continue or terminate in a controlled way.

2. Why Exception Handling is Required

Exceptions may occur because of invalid input, division by zero, accessing an invalid array index, using a null reference, converting invalid text to a number, file or resource problems and other runtime conditions. Exception handling separates normal program logic from error-handling logic.

3. Throwable Hierarchy

The root class for Java's exception mechanism is Throwable. Its major branches are Error and Exception. Error generally represents serious problems associated with the JVM or environment, while Exception represents conditions that applications commonly handle.

4. Checked and Unchecked Exceptions

Checked exceptions are exceptions that the compiler requires a program to handle or declare, such as IOException. Unchecked exceptions are RuntimeException subclasses and are generally detected during execution, such as ArithmeticException, NullPointerException and ArrayIndexOutOfBoundsException.

5. try Block

The try block contains statements that may cause an exception. A try block must be followed by at least one catch block or a finally block.

6. catch Block

A catch block handles an exception thrown from its associated try block. The catch parameter specifies the exception type that the block can handle. More specific exception types should be caught before broader types.

7. Multiple catch Blocks

A try block can have multiple catch blocks to handle different exception types. Each catch should represent a meaningful handling action. A general Exception catch should normally come after more specific catches.

8. finally Block

The finally block is used for cleanup code that should normally execute whether an exception occurs or not. It is commonly used for closing resources when try-with-resources is not being used.

9. throw

The throw statement explicitly creates or passes an exception object to the exception-handling mechanism. It is useful when a program detects an invalid condition and wants to signal it.

10. throws

The throws keyword is written in a method declaration to indicate that the method may pass specified exceptions to its caller. The caller can then handle the exception or declare it further.

11. Custom Exceptions

A programmer can create a custom exception by extending Exception for a checked exception or RuntimeException for an unchecked exception. Custom exceptions make application-specific error conditions clearer.

12. Exception Propagation

If an exception is not handled in the method where it occurs, it can propagate to the calling method. The process continues up the call stack until a matching catch block is found or the program terminates with an uncaught exception.

13. Nested try Blocks

A try block can appear inside another try block, method, loop or conditional structure. Nested try blocks should be used only when they make the handling structure clearer.

14. try-with-resources

The try-with-resources statement automatically closes objects that implement AutoCloseable, such as many file and stream classes. It reduces the need for explicit cleanup in finally blocks.

15. Common Exceptions

Common exceptions include ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException, InputMismatchException and IOException. The correct exception type helps the programmer understand and handle the actual problem.

16. Good Exception-Handling Practices

Catch the most specific exception that can be handled, avoid empty catch blocks, use meaningful messages, do not expose sensitive information, avoid using exceptions for ordinary control flow, and clean up resources safely.

17. Java Development and Execution

Compile Java source code with javac and run the compiled class with java. In VS Code, the Java Extension Pack or an equivalent Java extension can be used for editing, compiling and running programs. Use a JDK rather than only a JRE when compiling programs.

Part B – 20 Solved Practice Questions

1. Write a Java program to handle division by zero using try and catch.

Solution:

```
public class Q01_DivisionByZero {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;

        try {
            // Code that may produce ArithmeticException is placed in try.
            int result = a / b;
            System.out.println("Result = " + result);
        } catch (ArithmeticException e) {
            // Handle division by zero without terminating unexpectedly.
            System.out.println("Cannot divide by zero.");
        }

        System.out.println("Program continues normally.");
    }
}
```

The ArithmeticException is caught by the catch block, so the program continues after the exception.

2. Write a program to handle ArrayIndexOutOfBoundsException when accessing an invalid array index.

Solution:

```
public class Q02_ArrayException {
    public static void main(String[] args) {
        int[] marks = {80, 75, 90};
```

```
        try {
            // Valid indexes are 0, 1 and 2.
            System.out.println(marks[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index.");
        }
    }
}
```

The invalid index produces ArrayIndexOutOfBoundsException, which is handled by the catch block.

3. Write a program to demonstrate NullPointerException and handle it safely.

Solution:

```
public class Q03_NullPointer {
    public static void main(String[] args) {
        String name = null;

        try {
            // Calling a method through a null reference causes an exception.
            System.out.println(name.length());
        } catch (NullPointerException e) {
            System.out.println("The string reference is null.");
        }
    }
}
```

Calling length() on a null reference causes NullPointerException. The catch block handles the condition.

4. Write a program to handle NumberFormatException while converting a String into an integer.

Solution:

```
public class Q04_NumberFormat {
    public static void main(String[] args) {
        String value = "25A";

        try {
            // parseInt expects a valid integer representation.
            int number = Integer.parseInt(value);
            System.out.println("Number = " + number);
        } catch (NumberFormatException e) {
            System.out.println("Invalid integer value: " + value);
        }
    }
}
```

Because 25A is not a valid integer representation, Integer.parseInt() throws NumberFormatException.

5. Write a program using multiple catch blocks for different exception types.

Solution:

```
public class Q05_MultipleCatch {
    public static void main(String[] args) {
        int[] values = {10, 20, 30};

        try {
            int result = values[5] / 0;
            System.out.println(result);
        } catch (ArrayIndexOutOfBoundsException e) {
            // Handle an invalid array index.
            System.out.println("Array index is invalid.");
        } catch (ArithmeticException e) {
            // Handle arithmetic errors such as division by zero.
            System.out.println("Arithmetic error occurred.");
        }
    }
}
```

Only the first exception reached during execution is handled. Multiple catch blocks allow different exception types to be handled separately.

6. Write a program to demonstrate the finally block.

Solution:

```
public class Q06_Finally {
    public static void main(String[] args) {
        try {
            System.out.println("Inside try block");
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Exception handled.");
        } finally {
            // finally is used for cleanup or code that should normally run.
            System.out.println("Finally block executed.");
        }
    }
}
```

The finally block executes after the try/catch processing and is commonly used for cleanup.

7. Write a program to validate a student's marks and use throw to report invalid marks.

Solution:

```
public class Q07_Throw {
    static void validateMarks(int marks) {
        if (marks < 0 || marks > 100) {
            // Explicitly create an exception when the value is invalid.
            throw new IllegalArgumentException("Marks must be between 0 and 100");
        }
        System.out.println("Valid marks: " + marks);
    }

    public static void main(String[] args) {
        try {
            validateMarks(125);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

The throw statement explicitly signals an invalid condition. IllegalArgumentException is appropriate for an invalid argument value.

8. Write a program to demonstrate throws with a method that may generate a checked exception.

Solution:

```
import java.io.IOException;

public class Q08_Throws {
    static void readData() throws IOException {
        // The method declares that it may pass IOException to its caller.
        throw new IOException("Input operation failed");
    }

    public static void main(String[] args) {
        try {
            readData();
        } catch (IOException e) {
            System.out.println("Handled: " + e.getMessage());
        }
    }
}
```

throws declares a possible exception in the method signature, while the caller handles the exception with catch.

9. Write a program that demonstrates exception propagation from one method to another.

Solution:

```
public class Q09_Propagation {
    static void methodC() {
        // The exception starts in the deepest method.
        int result = 10 / 0;
        System.out.println(result);
    }

    static void methodB() {
        methodC();
    }

    static void methodA() {
        methodB();
    }

    public static void main(String[] args) {
        try {
            methodA();
        } catch (ArithmeticException e) {
            System.out.println("Exception handled in main.");
        }
    }
}
```

The exception moves up the call stack from methodC to methodB to methodA and is finally handled in main.

10. Write a program using a nested try block.

Solution:

```
public class Q10_NestedTry {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try started");

            try {
                // Inner try handles an operation with its own exception.
                int result = 20 / 0;
                System.out.println(result);
            } catch (ArithmeticException e) {
                System.out.println("Inner catch: division by zero");
            }

            System.out.println("Outer try continues");
        } catch (Exception e) {
            System.out.println("Outer catch: " + e.getMessage());
        }
    }
}
```

The inner catch handles the arithmetic exception, allowing the outer try block to continue.

11. Write a program to demonstrate the correct order of multiple catch blocks.

Solution:

```
public class Q11_CatchOrder {
    public static void main(String[] args) {
        try {
            int[] data = {1, 2};
            System.out.println(data[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
```

```
            // Specific exception must be checked before its broader parent.
            System.out.println("Specific array exception handled.");
        } catch (RuntimeException e) {
            // RuntimeException is broader than ArrayIndexOutOfBoundsException.
            System.out.println("Runtime exception handled.");
        }
    }
}
```

The more specific exception is placed first. A broader catch block placed first would make the later specific catch unreachable.

12. Create a custom checked exception named InvalidAgeException and use it to validate age.

Solution:

```
class InvalidAgeException extends Exception {
    // Custom checked exception for an application-specific condition.
    InvalidAgeException(String message) {
        super(message);
    }
}

public class Q12_CustomException {
    static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above.");
        }
        System.out.println("Eligible age: " + age);
    }

    public static void main(String[] args) {
        try {
            validateAge(16);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

A custom exception gives a meaningful application-specific name to an invalid condition. Extending Exception makes it checked.

13. Create an unchecked custom exception by extending RuntimeException and use it for invalid dosage input.

Solution:

```
class InvalidDosageException extends RuntimeException {
    InvalidDosageException(String message) {
        super(message);
    }
}

public class Q13_UncheckedCustomException {
    static void validateDosage(double dose) {
        if (dose <= 0) {
            // RuntimeException subclasses do not need to be declared with throws.
            throw new InvalidDosageException("Dosage must be greater than zero.");
        }
        System.out.println("Valid dosage: " + dose + " mg");
    }

    public static void main(String[] args) {
        try {
            validateDosage(-5);
        } catch (InvalidDosageException e) {
            System.out.println(e.getMessage());
```

```
        }
    }
}
```

This custom exception is unchecked because it extends RuntimeException. It is useful for invalid method arguments or application rules.

14. Write a program using try-with-resources to read a text file safely.

Solution:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Q14_TryWithResources {
    public static void main(String[] args) {
        // The resource is automatically closed when the try block ends.
        try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
            String line = reader.readLine();
            System.out.println("First line: " + line);
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

try-with-resources automatically closes the BufferedReader because it implements AutoCloseable, reducing manual cleanup code.

15. Write a program that uses multiple exceptions in a simple student result calculation.

Solution:

```
public class Q15_StudentResult {
    public static void main(String[] args) {
        String input = "90";
        int[] marks = {80, 85, 90};

        try {
            // String-to-integer conversion may cause NumberFormatException.
            int extra = Integer.parseInt(input);
            System.out.println("Extra value = " + extra);

            // Invalid index may cause ArrayIndexOutOfBoundsException.
            System.out.println("Mark = " + marks[5]);
        } catch (NumberFormatException e) {
            System.out.println("Invalid numeric input.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid marks index.");
        }
    }
}
```

Different catch blocks provide different responses for different failures. Only the exception actually reached during execution is handled.

16. Write a program that demonstrates a method declaring more than one checked exception with throws.

Solution:

```
import java.io.IOException;
import java.text.ParseException;

public class Q16_MultipleThrows {
    static void processData(boolean fileProblem, boolean parseProblem)
            throws IOException, ParseException {
        if (fileProblem) {
            throw new IOException("File operation failed");
        }
```

```
        if (parseProblem) {
            throw new ParseException("Data parsing failed", 0);
        }
        System.out.println("Data processed successfully.");
    }

    public static void main(String[] args) {
        try {
            processData(false, true);
        } catch (IOException e) {
            System.out.println("IO problem: " + e.getMessage());
        } catch (ParseException e) {
            System.out.println("Parsing problem: " + e.getMessage());
        }
    }
}
```

A method can declare multiple checked exception types. The caller can use separate catch blocks to handle them differently.

17. Create a custom InsufficientBalanceException for a bank withdrawal operation.

Solution:

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

public class Q17_BankException {
    static void withdraw(double balance, double amount)
            throws InsufficientBalanceException {
        if (amount > balance) {
            // Application-specific rule is represented by a custom exception.
            throw new InsufficientBalanceException("Insufficient balance.");
        }
        System.out.println("Withdrawal successful. Remaining balance = "
                + (balance - amount));
    }

    public static void main(String[] args) {
        try {
            withdraw(5000, 7000);
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Custom exceptions are useful when a program has domain-specific rules that should be communicated clearly to the caller.

18. Write a program showing that finally is used for cleanup even when an exception is handled.

Solution:

```
public class Q18_CleanupFinally {
    static void process() {
        try {
            System.out.println("Processing started.");
            int value = 100 / 0;
            System.out.println(value);
        } catch (ArithmeticException e) {
            System.out.println("Processing error handled.");
        } finally {
            // Cleanup code belongs here when try-with-resources is not suitable.
            System.out.println("Cleanup completed.");
```

```
        }
    }

    public static void main(String[] args) {
        process();
    }
}
```

The finally block is appropriate for cleanup actions that should normally occur after the try/catch processing.

19. Write a robust program to validate student marks using a custom exception, throw and catch.

Solution:

```
class InvalidMarksException extends Exception {
    InvalidMarksException(String message) {
        super(message);
    }
}

public class Q19_MarksValidation {
    static void checkMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
            throw new InvalidMarksException("Marks must be from 0 to 100.");
        }
        System.out.println("Accepted marks = " + marks);
    }

    public static void main(String[] args) {
        int[] testMarks = {85, 105, 72};

        for (int marks : testMarks) {
            try {
                // Each value is validated independently.
                checkMarks(marks);
            } catch (InvalidMarksException e) {
                System.out.println("Rejected: " + e.getMessage());
            }
        }
    }
}
```

The custom exception, throw, throws and catch work together to validate each input while allowing the loop to continue.

20. Develop a complete exception-handling workflow using input validation, custom exception, try, catch, finally and verification.

Solution:

```
import java.util.Scanner;

class InvalidAgeException20 extends Exception {
    InvalidAgeException20(String message) {
        super(message);
    }
}

public class Q20_CompleteExceptionWorkflow {
    static void validateAge(int age) throws InvalidAgeException20 {
        if (age < 18 || age > 100) {
            throw new InvalidAgeException20("Age must be between 18 and 100.");
        }
    }

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter age: ");
            String input = sc.nextLine();

            // Conversion can cause NumberFormatException.
            int age = Integer.parseInt(input);
            validateAge(age);
            System.out.println("Valid age: " + age);
        } catch (NumberFormatException e) {
            System.out.println("Enter a valid integer age.");
        } catch (InvalidAgeException20 e) {
            System.out.println("Invalid age: " + e.getMessage());
        } finally {
            // Close the Scanner after processing is complete.
            sc.close();
            System.out.println("Input resource closed.");
        }
    }
}
```

This program combines checked custom exceptions, throw, throws, multiple catch blocks, finally, input conversion and resource cleanup.

Part C – Revision and Java Exception-Handling Checklist

• Understand the difference between Error, Exception, checked exception and unchecked exception.
• Put only statements that may need handling inside the appropriate try block.
• Catch specific exceptions before broader exception types.
• Remember that throw is used to explicitly throw an exception, while throws declares possible exceptions in a method signature.
• Use finally for cleanup when appropriate; prefer try-with-resources for AutoCloseable resources.
• Do not leave catch blocks empty. Handle, report or rethrow the exception appropriately.
• Use custom exceptions when application-specific validation rules need meaningful exception types.
• Do not expose passwords, sensitive data or unnecessary internal details in exception messages.
• Compile and run each Java program separately using a suitable JDK.
• Verify the program with both valid and invalid inputs so that the exception-handling path is tested.